Multifactor Authentication Cheat Sheet¶

Introduction¶

Multifactor Authentication (MFA) requires evidence from at least two distinct authentication factors: something you know, something you have, and something you are. Location and other risk signals can inform authentication decisions, but do not substitute for an authentication factor, as explained in NIST SP 800-63B-4.

Factor	Examples
Something You Know	Passwords and PINs
Something You Have	OTP Tokens, U2F Tokens, Certificates, Smart Cards, SMS and Phone Calls (restricted)
Something You Are	Fingerprints, Facial Recognition, Iris Scans, Behavioral Biometrics

It should be noted that requiring multiple instances of the same authentication factor (such as needing both a password and a PIN) **does not constitute MFA** and offers minimal additional security. The factors used should be independent of each other and should not be able to be compromised by the same attack. While the following sections discuss the disadvantage and weaknesses of various different types of MFA, in many cases these are only relevant against targeted attacks. **Any MFA is better than no MFA**.

Advantages¶

The most common way that user accounts get compromised on applications is through weak, re-used or stolen passwords. Despite any technical security controls implemented on the application, users are liable to choose weak passwords, or to use the same password on different applications. As developers or system administrators, it should be assumed that users' passwords will be compromised at some point, and the system should be designed in order to defend against this.

MFA is by far the best defense against the majority of password-related attacks, including brute-force, credential stuffing and password spraying, with analysis by Microsoft suggesting that it would have stopped 99.9% of account compromises.

Disadvantages¶

The biggest disadvantage of MFA is the increase in management complexity for both administrators and end users. Many less technical users may find it difficult to configure and use MFA. Additionally, there are a number of other common issues encountered:

- Types of MFA that require users to have specific hardware can introduce significant costs and administrative overheads.
- Users may become locked out of their accounts if they lose or are unable to use their other factors.
- MFA introduces additional complexity into the application.
- Many MFA solutions add external dependencies to systems, which can introduce security vulnerabilities or single points of failure.
- Processes implemented to allow users to bypass or reset MFA may be exploitable by attackers.
- Requiring MFA may prevent some users from accessing the application.

Quick Recommendations¶

Exactly when and how MFA is implemented in an application will vary on a number of different factors, including the threat model of the application, the technical level of the users, and the level of administrative control over the users. These need to be considered on a per-application basis.

However, the following recommendations are generally appropriate for most applications, and provide an initial starting point to consider.

- Require some form of MFA for all users.
- Provide the option for users to enable MFA on their accounts using TOTP.
- Require MFA for administrative or other high privileged users.
- Implement a secure procedure to allow users to reset their MFA.
- Consider MFA as a service.

Implementing MFA¶

MFA is a critical security control, and is recommended for all applications. The following sections provide guidance on how to implement MFA, and the considerations that should be taken into account.

Regulatory and Compliance Requirements¶

Many industries and countries have regulations that require the use of MFA. This is particularly common in the finance and healthcare sectors, and is often required in order to comply with the General Data Protection Regulation (GDPR) in the European Union. It is important to consider these requirements when implementing MFA.

When to Require MFA¶

The most important place to require MFA on an application is when the user logs in. However, depending on the functionality available, it may also be appropriate to require MFA for performing sensitive actions, such as:

- Changing passwords or security questions.
- Changing the email address associated with the account.
- Disabling MFA.
- Elevating a user session to an administrative session.

If the application provides multiple ways for a user to authenticate these should all require MFA, or have other protections implemented. A common area that is missed is if the application provides a separate API that can be used to login, or has an associated mobile application.

One-Time Password (OTP) Handling and Storage¶

OTPs are authentication secrets and should be handled with password-like hygiene. While their security traits differ from long-lived passwords, improper handling can still lead to user account compromise.

At a minimum, OTP implementations SHOULD:

- Enforce a short time-to-live (TTL)
- Ensure OTPs are single use
- Apply strict attempt limits
- Invalidate the OTP on successful verification

OTP implementations SHOULD NOT:

- Log OTP values
- Store OTPs in long-term plaintext form

To further reduce risk and limit exposure, it is RECOMMENDED to:

- Generate OTPs using a cryptographically secure random number generator
- Consider 8-digit or longer codes where usability allows
- On "resend", generate a new OTP and overwrite the old record

Hashing OTPs¶

Hashing OTPs is still recommended, but for different reasons than password hashing. OTPs typically have a very small keyspace (for example, ~1 million possibilities for a 6-digit code), which means a database attacker can brute-force any OTP hash quickly.

As a result, hashing OTPs does not provide strong offline attack resistance in the way password hashing does.

However, hashing remains useful to:

- Prevent accidental disclosure via logs, metrics, or debugging tools
- Reduce blast radius if the database is briefly exposed during the OTP's validity window
- Enforce good secret-handling discipline and avoid plaintext storage by default

The goal is short-term exposure protection, not long-term cryptographic secrecy.

Improving User Experience¶

Risk Based Authentication¶

Having to frequently login with MFA creates an additional burden for users, and may cause them to disable MFA on the application. Risk based authentication can be used to reduce the frequency of MFA prompts, by only requiring MFA when the user is performing an action that is considered to be high risk. Some examples of this include:

- Requiring MFA when the user logs in from a new device or location.
- Requiring MFA when the user logs in from a location that is considered to be high risk.
- Using corporate IP ranges or geolocation as risk signals when deciding whether to require additional authentication.

Passkeys¶

Passkeys can provide phishing-resistant MFA by combining possession of the credential private key with local PIN or biometric verification. When using passkeys as MFA, require user verification and validate the returned user-verification flag on the server; a touch confirming user presence alone is not a second factor. The credential key pair is created during registration; authentication uses the existing private key to sign a challenge. For registration, verification, and recovery guidance, see the Passkey Security Cheat Sheet.

Failed Login Attempts¶

When a user enters their password, but fails to authenticate using a second factor, this could mean one of two things:

- The user has lost their second factor, or doesn't have it available (for example, they don't have their mobile phone, or have no signal).
- The user's password has been compromised.

There are a number of steps that should be taken when this occurs:

- Prompt the user to try another form of MFA.

- Allow the user to attempt to reset their MFA.
- Notify the user of the failed login attempt, and encourage them to change their password if they don't recognize it.
    - The notification should include the time, browser and geographic location of the login attempt.
    - This should be displayed next time they login, and optionally emailed to them as well.

Resetting MFA¶

One of the biggest challenges with implementing MFA is handling users who forget or lose their additional factors. There are many ways this could happen, such as:

- Re-installing a workstation without backing up digital certificates.
- Wiping or losing a phone without backing up OTP codes.
- Changing mobile numbers.

In order to prevent users from being locked out of the application, there needs to be a mechanism for them to regain access to their account if they can't use their existing MFA; however it is also crucial that this doesn't provide an attacker with a way to bypass MFA and hijack their account.

There is no definitive "best way" to do this, and what is appropriate will vary hugely based on the security of the application, and also the level of control over the users. Solutions that work for a corporate application where all the staff know each other are unlikely to be feasible for a publicly available application with thousands of users all over the world. Every recovery method has its own advantages and disadvantages, and these need to be evaluated in the context of the application.

Some suggestions of possible methods include:

- Providing the user with a number of single-use recovery codes when they first setup MFA.
- Requiring the user to setup multiple types of MFA (such as a digital certificate, OTP core and phone number for SMS), so that they are unlikely to lose access to all of them at once.
- Mailing a one-use recovery code (or new hardware token) to the user's registered address.
- Requiring the user contact the support team and having a rigorous process in place to verify their identity.
- Requiring another trusted user to vouch for them.

Changing MFA Factors¶

Users may need to update their authentication factors, such as changing a phone number, migrating to a new authenticator app, or replacing a lost hardware token. Because attackers can exploit this process to take over accounts, it must be strictly secured.

Best practices include:

- Require reauthentication with an existing enrolled factor before allowing changes.
- Do not rely solely on the active session, as it may be hijacked.
- Treat factor replacement as a high-risk action and apply risk-based checks (e.g., new device, unusual location).
- Notify the user through out-of-band channels (such as email or push notification) whenever an MFA factor is changed.
- Consider applying delays or step-up verification for high-value accounts.

This ensures that even if a session is compromised, attackers cannot silently replace the user's MFA factors and lock the legitimate user out.

Consider Using a Third Party Service¶

There are a number of third party services that provide MFA as a service. These can be a good option for applications that don't have the resources to implement MFA themselves, or for applications that require a high level of assurance in their MFA. However, it is important to consider the security of the third party service, and the implications of using it. For example, if the third party service is compromised, it could allow an attacker to bypass MFA on all of the applications that use it.

Something You Know¶

Knowledge-based, the most common type of authentication is based on something the users knows - typically a password. The biggest advantage of this factor is that it has very low requirements for both the developers and the end user, as it does not require any special hardware, or integration with other services.

Passwords and PINs¶

Passwords and PINs are the most common form of authentication due to the simplicity of implementing them. The Authentication Cheat Sheet has guidance on how to implement a strong password policy, and the Password Storage Cheat Sheet has guidance on how to securely store passwords. Most multifactor authentication systems make use of a password, as well as at least one other factor.

Pros¶

- Simple and well understood.
- Native support in every authentication framework.

- Easy to implement.

Cons¶

- Users are prone to choosing weak passwords.
- Passwords are commonly re-used between systems.
- Susceptible to phishing.

Security Questions¶

Security questions are no longer recognized as an acceptable authentication factor per NIST SP 800-63. Account recovery is just an alternate way to authenticate so it should be no weaker than regular authentication.

Pros¶

- None that are not also present in passwords.

Cons¶

- No longer recognized as an acceptable authentication factor.
- Questions often have easily guessable answers.
- Answers to questions can often be obtained from social media or other sources.
- Questions must be carefully chosen so that users will remember answers years later.
- Susceptible to phishing.

Something You Have¶

Possession-based authentication is based on the user having a physical or digital item that is required to authenticate. This is the most common form of MFA, and is often used in conjunction with passwords. The most common types of possession-based authentication are hardware and software tokens, and digital certificates. If properly implemented then this can be significantly more difficult for a remote attacker to compromise; however it also creates an additional administrative burden on the user, as they must keep the authentication factor with them whenever they wish to use it.

One-Time Password Tokens¶

One-Time Password (OTP) tokens are a form of possession-based authentication, where the user is required to submit a constantly changing numeric code in order to authenticate. The most common of which is Time-based One-Time Password (TOTP) tokens, which can be both hardware and software based.

Hardware OTP Tokens¶

Hardware OTP Tokens generate a constantly changing numeric codes, which must be submitted when authenticating. Most well-known of these is the RSA SecureID, which generates a six digit number that changes every 60 seconds.

Pros¶

- As the tokens are separate physical devices, they are almost impossible for an attacker to compromise remotely.
- Tokens can be used without requiring the user to have a mobile phone or other device.

Cons¶

- Deploying physical tokens to users is expensive and complicated.
- If a user loses their token it could take a significant amount of time to purchase and ship them a new one.
- Some implementations require a backend server, which can introduce new vulnerabilities as well as a single point of failure.
- Stolen tokens can be used without a PIN or device unlock code.
- Susceptible to phishing (although short-lived).

Software OTP Tokens¶

A cheaper and easier alternative to hardware tokens is using software to generate Time-based One-Time Password (TOTP) codes. This would typically involve the user installing a TOTP application on their mobile phone, and then scanning a QR code provided by the web application which provides the initial seed. The authenticator app generates a numeric code using a configured time step; RFC 6238 recommends a default of 30 seconds. The app and server must use the same time-step value.

Most websites use standardized TOTP tokens, allowing the user to install any authenticator app that supports TOTP. However, a small number of applications use their own variants of this (such as Symantec), which requires the users to install a specific app in order to use the service. This should be avoided in favor of a standards-based approach.

Pros¶

- The absence of physical tokens greatly reduces the cost and administrative overhead of implementing the system.
- When users lose access to their TOTP app, a new one can be configured without needing to ship a physical token to them.
- TOTP is widely used, and many users will already have at least one TOTP app installed.
- A screen lock reduces theft risk but does not guarantee protection if the phone is stolen while unlocked. Enable authenticator-app access protection where available; for example, Google Authenticator’s Privacy Screen requires device verification before app use.

Cons¶

- TOTP apps are usually installed on mobile devices, which are vulnerable to compromise.
- The TOTP app may be installed on the same mobile device (or workstation) that is used to authenticate.
- Users may store the backup seeds insecurely.
- Not all users have mobile devices to use with TOTP.
- If the user's mobile device is lost, stolen or out of battery, they will be unable to authenticate.
- Susceptible to phishing (although short-lived).

Universal Second Factor¶

Hardware U2F tokens

Universal Second Factor (U2F) is a standard for USB/NFC hardware tokens that implement challenge-response based authentication, rather than requiring the user to manually enter the code. This would typically be done by the user pressing a button on the token, or tapping it against their NFC reader. The most common U2F token is the YubiKey.

Pros¶

- U2F tokens are resistant to phishing since the private key never leaves the token.
- Users can simply press a button rather than typing in a code.
- As the tokens are separate physical devices, they are almost impossible for an attacker to compromise remotely.
- U2F is natively supported by a number of major web browsers.
- U2F tokens can be used without requiring the user to have a mobile phone or other device.

Cons¶

- As with hardware OTP tokens, the use of physical tokens introduces significant costs and administrative overheads.
- Stolen tokens can be used without a PIN or device unlock code.
- As the tokens are usually connected to the workstation via USB, users are more likely to forget them.

Certificates¶

Digital certificates are files that are stored on the user's device which are automatically provided alongside the user's password when authenticating. The most common type is X.509 certificates more commonly known as client certificates. Certificates are supported by all major web browsers, and once installed require no further interaction from the user. The certificates should

be linked to an individual's user account in order to prevent users from trying to authenticate against other accounts.

Pros¶

- There is no need to purchase and manage hardware tokens.
- Once installed, certificates are very simple for users.
- Certificates can be centrally managed and revoked.
- Resistant to phishing.

Cons¶

- Using digital certificates requires a backend Private Key Infrastructure (PKI).
- Installing certificates can be difficult for users, particularly in a highly restricted environment.
- Enterprise proxy servers which perform SSL decryption will prevent the use of certificates.
- The certificates are stored on the user's workstation, and as such can be stolen if their system is compromised.

Smart Cards¶

Smartcards are credit-card size cards with a chip containing a digital certificate for the user, which is unlocked with a PIN. They are commonly used for operating system authentication, but are rarely used in web applications.

Pros¶

- Stolen smartcards cannot be used without the PIN.
- Smartcards can be used across multiple applications and systems.
- Resistant to phishing.

Cons¶

- Managing and distributing smartcards has the same costs and overheads as hardware tokens.
- Smartcards are not natively supported by modern browsers, so require third party software.
- Although most business-class laptops have smartcard readers built-in, home systems often do not.
- The use of smartcards requires backend PKIs.

SMS Messages and Phone Calls¶

[!WARNING] NIST SP 800-63B-4 designates SMS and PSTN-delivered codes as a *restricted* authenticator because of SS7 interception, SIM-swap, and number-porting attacks. Do not use

SMS for high-value or PII-handling applications. Where it is the only available factor, document the risk acceptance, enforce per-account rate limits, monitor for SIM-swap signals, and plan migration to TOTP, push notifications, or WebAuthn/FIDO2.

SMS messages or phone calls can be used to provide users with a single-use code that they must submit as an additional factor. Due to the risks posed by these methods, they should not be used to protect applications that hold Personally Identifiable Information (PII) or where there is financial risk. e.g. healthcare and banking. NIST SP 800-63B classifies these as restricted authenticators and discourages their use for applications containing PII.

Pros¶

- Relatively simple to implement.
- Requires user to link their account to a mobile number.

Cons¶

- Requires the user to have a mobile device or landline.
- Require user to have signal or internet access to receive the call or message.
- Calls and SMS messages may cost money to send need to protect against attackers requesting a large number of messages to exhaust funds.
- Susceptible to SIM swapping attacks.
- SMS messages may be received on the same device the user is authenticating from.
- Susceptible to phishing.
- SMS may be previewed when the device is locked.
- SMS may be read by malicious or insecure applications.

Email¶

Email verification requires that the user enters a code or clicks a link sent to their email address. There is some debate as to whether email constitutes a form of MFA, because if the user does not have MFA configured on their email account, it simply requires knowledge of the user's email password (which is often the same as their application password). However, it is included here for completeness.

Pros¶

- Very easy to implement.
- No requirements for separate hardware or a mobile device.

Cons¶

- Relies entirely on the security of the email account, which often lacks MFA.
- Email passwords are commonly the same as application passwords.
- Provides no protection if the user's email is compromised first.
- Email may be received by the same device the user is authenticating from.

- Susceptible to phishing.

Something You Are¶

Inherence-based authentication is based on the physical attributes of the user. This is less common for web applications as it requires the user to have specific hardware, and is often considered to be the most invasive in terms of privacy. However, it is commonly used for operating system authentication, and is also used in some mobile applications.

Biometrics¶

The are a number of common types of biometrics that are used, including:

- Fingerprint scans
- Facial recognition
- Iris scans
- Voice recognition

Pros¶

- Well-implemented biometrics are hard to spoof, and require a targeted attack.
- Fast and convenient for users.

Cons¶

- Manual enrollment is required for the user.
- Custom (sometimes expensive) hardware is often required to read biometrics.
- Privacy concerns: Sensitive physical information must be stored about users.
- If compromised, biometric data can be difficult to change.
- Hardware may be vulnerable to additional attack vectors.

Somewhere You Are¶

Location can inform access restrictions and risk-based authentication decisions. It is not an independent authentication factor: a password combined with a trusted IP address or location does not constitute MFA. NIST SP 800-63B-4 explicitly distinguishes these risk signals from authentication factors.

Source IP Address¶

The source IP address the user is connecting from can be used as a risk signal or access restriction, typically in an allow-list based approach. This could either be based on a static list (such as corporate office ranges) or a dynamic list (such as previous IP addresses the user has authenticated from).

Pros¶

- Very easy for users.
- Requires minimal configuration and management from administrative staff.

Cons¶

- Doesn't provide any protection if the user's system is compromised.
- Doesn't provide any protection against rogue insiders.
- Trusted IP addresses must be carefully restricted (for example, if the open guest Wi-Fi uses the main corporate IP range).

Geolocation¶

Rather than using the exact IP address of the user, the geographic location that the IP address is registered to can be used. This is less precise, but may be more feasible to implement in environments where IP addresses are not static. A common usage would be to require additional authentication factors when an authentication attempt is made from outside of the user's normal country.

Pros¶

- Very easy for users.

Cons¶

- Doesn't provide any protection if the user's system is compromised.
- Doesn't provide any protection against rogue insiders.
- Easy for an attacker to bypass by obtaining IP addresses in the trusted country or location.
- Privacy features such as Apple's iCloud Private Relay and VPNs can make this less accurate.

Geofencing¶

Geofencing is a more precise version of geolocation, which allows the user to define a specific area in which they are allowed to authenticate. This is often used in mobile applications, where the user's location can be determined with a high degree of accuracy using geopositioning hardware like GPS.

Pros¶

- Very easy for users.
- Provides a high level of protection against remote attackers.

Cons¶

- Doesn't provide any protection if the user's system is compromised.
- Doesn't provide any protection against rogue insiders.
- Doesn't provide any protection against attackers who are physically close to the trusted location.

Something You Do¶

Behavioral biometrics, such as keystroke patterns or gait, are a form of "something you are," not a separate authentication factor. Under NIST's biometric requirements, biometric comparison must be combined with an authenticated physical authenticator. General activity signals, such as login times or navigation patterns, can inform risk decisions but do not by themselves constitute MFA.

Behavioral Profiling¶

Behavioral profiling is based on the way the user interacts with the application, such as the time of day they log in, the devices they use, and the way they navigate the application. This is rapidly becoming more common in web applications when combined with Risk Based Authentication and User and Entity Behavior Analytics (UEBA) systems.

Pros¶

- Doesn't require user interaction.
- Can be used to continuously authenticate the user.
- Combines well with other factors to increase the level of assurance in the user's identity.

Cons¶

- Early implementations of behavioral profiling were often inaccurate and caused a significant number of false positives.
- Requires large amounts of data and processing power to analyze the user's behavior.
- May be difficult to implement in environments where the user's behavior is likely to change frequently.

Keystroke & Mouse Dynamics¶

Keystroke and mouse dynamics are based on the way the user types and moves their mouse. For example, the time between key presses, the time between key presses and releases, and the speed and acceleration of the mouse. Largely theoretical, and not widely used in practice.

Pros¶

- Can be used without requiring any additional hardware.
- Can be used without requiring any additional interaction from the user.
- Can be used to continuously authenticate the user.

- Can be used to detect when the user is not the one using the system.
- Can be used to detect when the user is under duress.
- Can be used to detect when the user is not in a fit state to use the system.

Cons¶

- Unlikely to be accurate enough to be used as a standalone factor.
- May be spoofed by AI or other advanced attacks.

Gait Analysis¶

Gait analysis is based on the way the user walks using cameras and sensors. They are often used in physical security systems, but are not widely used in web applications. Mobile device applications may be able to use the accelerometer to detect the user's gait and use this as an additional factor, however this is still largely theoretical.

Pros¶

- Very difficult to spoof.
- May be used without requiring any additional interaction from the user.

Cons¶

- Requires specific hardware to implement.
- Use outside of physical security systems is not widely tested.

Adaptive or Risk-Based Authentication¶

Adaptive (or Risk-Based) Authentication adjusts authentication requirements dynamically based on the context of the login attempt. This technique helps improve user experience while strengthening security by applying additional verification steps only when risk is elevated.

Common signals used to determine risk include:

- Geolocation and IP reputation
- Device fingerprinting
- Time of access (e.g., 3 AM login)
- Behavioral biometrics (e.g., typing speed or mouse movements)
- Known compromised credentials

If risk is detected, the system may:

- Prompt for an additional factor (e.g., OTP)
- Enforce re-authentication
- Deny access and trigger alerting or account protection flows

For more details on when to trigger reauthentication after high-risk events—such as account recovery or suspicious activity—see the Reauthentication After Risk Events section in the Authentication Cheat Sheet.

This method is widely used in modern authentication systems to balance usability and security. However, developers must ensure that risk signals cannot be spoofed and that fallback mechanisms are not weaker than the primary MFA methods.

Example Use Case: A user logs in from a trusted device in a usual location — no additional prompt is needed. But if they log in from a new country using a Tor exit node, the system requires SMS verification or triggers an account lock until further verification.

MFA Attack Patterns and Mitigations¶

Attackers increasingly target weaknesses in MFA deployments and authentication workflows rather than attempting to defeat MFA itself. The following sections describe common attack patterns and recommended mitigations based on guidance from NIST SP 800-63B, CISA's Implementing Phishing-Resistant MFA, the FIDO Alliance Specifications, and OAuth 2.0 Security Best Current Practice (RFC 9700).

General Mitigation: Primary Phishing-Resistant Control¶

Prefer phishing-resistant authenticators (FIDO2/WebAuthn), which bind authentication to the legitimate origin and resist credential theft, MFA fatigue, and reverse-proxy phishing. See the Passkeys section for additional guidance.

MFA Fatigue (Push Notification Bombing)¶

Attackers repeatedly send MFA push notifications, often combined with social engineering, hoping the user eventually approves one.

Mitigations¶

- Require challenge-response push authentication (for example, number matching) to prevent blind approval of authentication requests.
- Rate-limit or cap push notifications to prevent repeated prompt abuse.
- Monitor for anomalous authentication activity, such as multiple push prompts in a short period or new-device token reuse.

Real-Time Phishing Using Reverse Proxies¶

Attackers use reverse-proxy phishing frameworks (such as Evilginx, Modlishka, and Muraena) to present convincing copies of legitimate login pages. These frameworks relay authentication

traffic between the user and the legitimate service, allowing attackers to capture credentials and session tokens in real-time.

Mitigations¶

- Monitor for anomalous authentication and session activity that may indicate credential or session compromise (for example, impossible travel, new ASN, or sudden MFA method changes).
- Use phishing-resistant authenticators (see Passkeys section), which bind authentication to the legitimate origin and are resistant to real-time phishing attacks.

SIM Swap and Phone Number Takeover¶

Attackers convince a telecommunications provider to transfer a victim's phone number, allowing interception of SMS or voice-based one-time passwords.

Mitigations¶

- Encourage carrier account PINs or port-out protection where available to reduce unauthorized number transfers.
- Monitor for unexpected phone-number changes or SIM replacement events.
- Prefer phishing-resistant authenticators or TOTP instead of SMS/voice OTP.
- See also: SMS Messages and Phone Calls.

Device Binding Bypass¶

Attackers attempt to bypass device-based authentication by extracting exportable cryptographic keys or replaying cloned device attributes when authenticators are not hardware-protected.

Mitigations¶

- Prefer hardware-backed, non-exportable cryptographic keys (for example, platform authenticators backed by TPM, Secure Enclave, or Android StrongBox).
- Validate authenticator attestation where required by organizational policy.

MFA Downgrade Attacks¶

Attackers attempt to force authentication through legacy protocols or authentication flows that do not enforce the same MFA requirements as modern authentication methods. Weak fallback mechanisms or legacy authentication endpoints can allow users to authenticate with lower-assurance factors than intended.

Mitigations¶

- Disable legacy authentication protocols and endpoints that cannot enforce MFA consistently.

- Prevent fallback from phishing-resistant authenticators to lower-assurance authentication methods unless required by a documented security policy.
- Follow OAuth 2.0 Security Best Current Practice (RFC 9700) to prevent OAuth protocol-level downgrade and mix-up attacks.